

AI社員に任せたら10点中6点。スキル化で8点、アプリ化で10点 あなたは、どれにする？

AI活用比較 — 同じ YouTube 要約を（3方式 [まるなげ・スキル化・アプリ化] × 2モデル [Composer・Claude Opus]）× 3 で、要約品質・実行時間・コストを18回測った

簡易結論

Composer のアプリ化が、3指標を総合すると最良だった。品質だけ見れば Claude + スキルも同点10点だが、時間・コストでは Composer + アプリが上。

簡易提案

まるなげから一步進みたいなら、**スキル化 →（繰り返すなら）アプリ化** が現実的な順番です。本稿の §7 に、いつもの依頼文の前に貼るだけのプロンプト例（スキル化用・アプリ化用）を載せています。測定結果の再現だけでなく、明日から試せる手順として使ってください。

詳細は以下です。

タイトルの6・8・10点は、**Composer 2.5** における各方式の最良試行の代表値（同一71分動画・品質10点満点）。Claude ではまるなげ7点・スキル10点・アプリ8点。

1. キッカケ（なぜ測るのか）

最近、ずっと引っかかっていることがあります。

SNSでよく見る「AI社員」「仕事をAIにまるなげ」。正直、あまりピンとこなかった。

仕事は楽になるかもしれない。でも、**コストはどうなの？**

普通、業務効率化ならコストも下げないと意味がない。人間の作業時間が減ったのはわかる。

見るべきなのは、たぶんここです。

- 削減した時間 × 自分の単価
- その時間を作るために AI に払う単価
- **（見落としがち）** 結果を確認・直す時間 × 自分の単価

AI側が小さければ得だ。でも本当にそうになっているのかは、投稿からは見えにくい。

「AI社員」という言葉は、うまく聞こえる一方で、**仕事の渡し方**までは語られないことが多い気がします。丸投げなのか、手順を決めているのか、仕組みを作っているのか——ここが違えば、時間も品質もコストも変わるはずです。

だから同じ土俵で同じ処理をさせて、測定する。感想ではなく、**時間・品質・トークンコスト**で比較しようと思った。

具体的には、1つの YouTube URL から字幕データを取得して要約する作業を、**まるなげ / スキル / アプリ** の3つのやり方で比較しました。

注: 全員がこの作業を日常業務にしているわけではありません。ただ、情報整理系の仕事（議事録・リサーチ・報告書など）の縮図として、再現しやすいタスクだと考えました。

なお、プロンプトだけ渡したまるなげは「依頼して結果が返った」ように見えますが、裏ではエージェントが毎回その場でスクリプトやコマンドを動かしています。手順を固定するのがスキル化、機械でできる部分をコードに切り出すのがアプリ化——本稿の3方式の出発点はここです（詳しくは §3-1）。

2. 結論（実測後）

先に書いていた想定はこうでした。

同じ仕事なら、高単価モデルでなくても Cursor + Composer2.5 で十分戦えるはず。高いモデル必須、みたいな空気は過大評価だと思っている。

測った結果、「高いモデル必須」への反論は当たった。

ただし主語を忘れていた——正しくは、

同じ仕事なら、高単価モデルでなくても **Composer 2.5 + アプリ化** で十分戦える。高いモデル必須、みたいな空気は過大評価だと思っている。

まるなげの Composer 2.5 だけでは品質6点で届かず、**勝ち筋はモデル選び単体ではなく、アプリ化まで含めた構成**だった。

2-1. 一言でいうと

言い方	実測から言えること
「高いモデルが必須」	必須ではない （Composer + アプリで満点10点）
「安いモデルで丸投げすれば十分」	十分ではない （まるなげは両モデルで最下位）
「AI社員に任せれば楽」	楽にはなる。ただし、出てきた結果をちゃんと検証していますか？ 検証を省くと、あとで困ることがある

まるなげは手は動かさなくてよいが品質6〜7点で、**検証コスト**が別途乗りやすい（\$5）。「楽になった分、誰が正しさを担保するか」——設計コストが見えにくいのが「AI社員」マーケティングの盲点でもある。品質の争点は方式、時間とコストの争点はモデル——数値は §2-2、読み方は §2-3。

2-2. 各方式の最良試行（同一動画・71分・品質10点満点）

対象: https://www.youtube.com/watch?v=koBLOf-53_g
品質評価: 字幕照合ベースの5観点（A〜E）各0〜2点、合計10点満点。観点の定義は [付録A](#) を参照。

方式×モデル×3指標（各組み合わせの最良試行）

形式は **品質 / 時間 / コスト** の順。★は本件の総合最良（Composer + アプリ）。

	まるなげ	スキル	アプリ
Composer 2.5	6点 / 1分18秒 / \$0.29	8点 / 1分00秒 / \$0.09	10点 / 47秒 / \$0.09 ★

	まるなげ	スキル	アプリ
Claude Opus 4.8	7点 / 4分59秒 / \$1.43	10点 / 3分41秒 / \$1.08	8点 / 2分34秒 / \$1.32

インタラクティブ版（Cursor Canvas・作成済みなら）：付録の [1枚図リンク](#) を参照。上表が静的版です。

何が差をつけやすいか:

観点	差が出るポイント	今回の目安（実測）
品質	方式（まるなげ・スキル・アプリ）	同一モデル内で 最大4点差 （例: Composer まるなげ6点 → アプリ10点）
時間	モデル（Composer / Claude）	同一方式で 約3〜4倍 （例: アプリ 47秒 vs 2分34秒）
コスト	モデル（Composer / Claude）	同一方式で 約10〜15倍 （例: アプリ 約\$0.09 vs 約\$1.32）

本件の結論: Composer + アプリが品質・時間・コストの総合で最良（Claude + スキルも品質10点だが、時間約5倍・コスト約12倍）。1/12・1/5の読み方は \$2-3。

- 秒数・ドルは各方式の**最良試行**。全18試行の早見表は付録A。コストは [Cursor 公式単価](#) の参考見積（実測請求はプラン内 Included）。
- 追記（2026年7月）**：First-party プール（Composer 2.5 等）の含まれる利用量が2倍に——\$2.6 参照。

2-3. コストと時間の差（補足）

指標	今回の実測	一般化できること	今回に限ったこと
コスト（約1/12）	Composer アプリ 約\$0.09 vs Claude スキル 約\$1.08	単価差（First-party vs API）は構造的。短い文章でも Composer 側が安い傾向は残る	トークン総量が多いほど差は開きやすい
時間（約1/5）	47秒 vs 3分41秒	Composer の方が速い傾向は残る	差の倍率は、今回ほどには出ない可能性が高い

時間差が大きく見えた理由は、おもに次の3点です。

- 動画が71分と長い** — 8セグメント分の要約があり、エージェントの読み込み・生成時間が両方に乗る
- Claude Opus は1トークンあたりの応答が遅い** — 文章量にほぼ比例して待ち時間が伸びる
- アプリの前処理（約10〜12秒）は固定** — 全体が短くなると、この固定コストの比重が大きくなる

ざっくり言うと、処理時間は次のイメージです。

$$\text{総時間} \div \text{固定コスト（起動・前処理など）} + \text{文章量に比例する部分} \times \text{モデルの速さ}$$

文章量が減ると、後ろの「比例する部分」が縮む。両モデルとも短くなる一方、**固定コストはあまり減らないので、47秒 vs 221秒（約1/5）ほどの開きは、短いタスクでは縮まりやすい**と考えてよいです。

タスクのイメージ	時間差のイメージ
今回（71分・8セグメント）	約 1/5 （実測どおり。差が際立つ）
中程度（10～15分・1～2セグメント）	1/3 前後 に縮む可能性
短い（数分・要約数行）	1/2 前後 まで縮む可能性（固定コストが支配的）

逆に、コスト面の約1/12は、文章量が減っても単価差が残るので、そこまで縮まない可能性が高いです（短くなれば絶対額は両方とも小さくなるが、比率は比較的維持されやすい）。

「約1/12のコスト」は今回の条件でも、やや短いタスクでも、おおむね言える。
「約1/5の時間」は今回の長尺・多セグメント例で際立った数字。文章量が減れば、この倍率差は小さくなると言てよい。

2-4. 用途別のおすすめ（本件限定）

優先したいこと	おすすめ
品質・速度・コストのバランス	Composer + アプリ （満点・今回47秒・約9セント）
引用の正確さ・監査向け	Claude + スキル （満点・忠実性で最良）
月額を増やさず外部APIで回す	Gemini API + アプリ （別枠・最良8点・無料枠。要試行選別・検証）
とにかく手を動かしたくない	まるなげ（ただし品質6～7点。事実確認は必須）

※47秒は71分動画での実測。短い素材では絶対時間もモデル間の倍率差も縮みます（2-3 参照）。

2.5. 別枠（Gemini API + アプリ） — 要点

本線（Cursor 上の Composer / Claude）とは **商品も課金体系も違う** 別枠です。同一動画・同一パイプラインで3試行した結果、**最良8点・ばらつき 8 / 7 / 3**（試行3は不採用）。無料枠は魅力的だが、本線の Composer + アプリ（10点）には届かず、**試行選別と検証が要る**——が要点です。詳細は下の折りたたみ、または別記事化予定（[4_比較結果/比較結果_gemini/](#)）。

▶ **\$2.5 詳細を読む**（Gemini 3試行・本線比較・パイプライン改善・再実行コマンド）

2.6. 追記：Cursor の利用枠2倍化で、差はさらに開く

先日、Cursor から **Grok 4.5** と **Composer 2.5** について、利用枠を **2倍** にしたと発表がありました。

ここでいう「2倍」は、プランに含まれる **First-party models プール**（旧 Auto + Composer プール）の使える量が増えた、という意味です。言い換えると **値下げ** です。

項目	内容
対象	Composer 2.5、Grok 4.5 など First-party モデル
変わったこと	含まれる利用量が2倍（同じ月額でより多く使える）
変わっていないこと	トークン単価表（例: Composer 入力 \$0.50/M、出力 \$2.50/M）

項目	内容
本実験との関係	最良構成だった Composer + アプリ が、さらにコスト面で有利に

一方、本実験の比較相手である **Claude Opus 4.8** は **API プール**（サードパーティ）から消費されます。単価が高いうえ、今回の2倍化の対象外。同じ月額で **Composer 側だけ実行回数の余地が広がる**——§2-3 のコスト差（約1/12）に加えて、プラン内運用では実質的な差がさらに開きやすい、というのが追記の要点です。

この追記が意味すること

1. **Composer + アプリ** 路線の合理性が、実験後にさらに強くなった

品質・コストで既に有利だった構成が、プラン内では同じ月額でより多く回せる。

2. 「高いモデルを選べば安心」は、コスト面ではますます厳しい

忠実性が必要な場面では Claude + スキルが有効、という実測結論は変わらない。ただし **同じプラン内で何本回すか** を考えると、Composer 側の余裕が増える。

3. **Grok 4.5** も同じプール

今回は Composer 2.5 で測ったが、Grok 4.5 も First-party 枠に入る。要約・リサーチ系タスクでは、今後こちらでも同様の比較ができそう（別途検証予定）。

4. 注意: 2倍化は「品質保証」ではない

利用枠が増えても、まるなげの幻覚リスクは消えない。安くたくさん回せるからこそ、方式（スキル／アプリ）の設計がより重要になる。

同じ処理を繰り返し使うなら、一度立ち止まってほしい

今回の数字は、あくまで1本の動画・1種類のタスクでの話です。ただ、次のような使い方をしているなら、モデル選びを見直す余地は大きいかもしれません。

- 同じような仕事を、月に何度も回している
- いま **Claude Opus** など高単価モデルをデフォルトにしている
- 毎回まるなげ方式に近い使い方をしている

本件では **Composer 2.5 + アプリ化** が、品質・時間・コストのバランスでいちばんよかった。単価差だけ見ても Claude 路線の約1/12の見積で、しかも今回の利用枠2倍化で **Composer 2.5 は同じ月額のまま、おおよそ2倍まで実行できる余地が広がった**（First-party プールの話。§2-3 参照）。

繰り返し運用なら、アプリ化の初期設計コストはだんだん償却されていきます（\$5）。だから、「いまの高コストモデルのままでいいのかな」と、一度だけでいいので自分に問いかけてみてほしい——そう思います。

もちろん、引用の厳密さや監査が最優先なら Claude + スキルが有力、というのは §4-4 のとおりです。全部を安いモデルに寄せろ、という話ではなく、「毎回のデフォルト」が本当に最適かどうか、を確認する、くらいの温度感です。

出典: [Cursor Grok 4.5 発表](#)、[Models & Pricing](#)

First-party プールと API プールの違いは、Cursor フォーラムの公式回答でも確認できます。

3. 何を比較するか（ルール）

比較のルールは以下の通りです。

やり方は3つ

方式	内容	「AI社員」との対応
まるなげ	長いプロンプトを貼って全部任せる	SNS的な「丸投げ」に近い
スキル	Cursor Skill で手順・品質基準を標準化	業務マニュアル化
アプリ	Python で前処理を機械化、AIは要約のみ	業務システム化

3-1. 見えていることと、裏で起きていること

プロンプトだけ渡してまるなげすると、人間からは「依頼して結果が返ってきた」ように見えます。

しかし Cursor のようなエージェント型AIでは、裏ではだいたい次のことをしています。

- 1. プロンプトから必要な作業を分解する
- 2. その場でスクリプトやコマンドを書き、ファイルを読み書きする
- 3. 実行結果をまとめて、人間に返す

つまり「**プロンプト → 結果**」に見えても、**中身は毎回その場しのぎの自動化**です。手順が固定されないの
で、試行ごとにブレやすい——今回のまるなげが品質6〜7点帯に留まった背景のひとつだと考えています。

ここから先は、裏側の作業を段階的に「仕組み」に落としていく話です。

段階	固定するもの	効くこと
スキル化	手順・品質基準・出力形式	毎回のやり方を安定させる。まるなげより確実に
アプリ化	字幕取得・整形など機械でできる部分	再現性のあるコードに切り出し、 その部分のAIコストをゼロに 、処理時間を短縮し、 AIに渡す材料の精度 を上げる

本稿の「アプリ」は、あらかじめ Python パイプラインを用意した方式を指します。まるなげのときエージェントがその場でやっている処理の、**あらかじめ作っておく版**——と捉えると、3方式の違いが整理しやすいです。

モデルは2つ

- Composer 2.5 (Fast でない方)
- Claude Opus 4.8 low Thinking

見るのは3つ

- 1. かかった時間（プロンプト入力直前〜結果出力まで）
- 2. 要約の品質（字幕照合・5観点スコア。定義は [付録A](#)）
- 3. トークン量（ドル換算）

単価の参照: <https://cursor.com/ja/docs/models-and-pricing>

プールの違い（コスト比較で重要）

プール	主なモデル	本実験での位置づけ
First-party models	Composer 2.5、Grok 4.5、Auto	安い・2026年7月に利用枠2倍化
API（サードパーティ）	Claude Opus、GPT など	高単価・別枠
	Composer 2.5	Claude Opus 4.8
入力	\$0.50/M	\$5.00/M
キャッシュ書き込み	—	\$6.25/M
キャッシュ読み取り	\$0.20/M	\$0.50/M
出力	\$2.50/M	\$25.00/M
プラン内の扱い	First-party プール	API プール

その他のルール

- 同じ YouTube URL で固定
- 方式 × モデルごとに各3回（計18試行）
- 本番判断は各組み合わせの**最高品質試行**を代表とする（品質があつての時間・コスト）

本件の限界（読み方の前提）

- 対象は**1本の動画・1種類のタスク**（71分・YouTube字幕要約）。他ジャンル・短尺への一般化は未検証。
- 品質は**字幕照合ベースの5観点**（付録A）で、実験者が **要約比較プロンプト.md** に沿って採点。第三者レビューではない。
- **アプリ化の初期設計コスト**（パイプライン開発時間）は経済式（\$5）に概念として入れたが、本比較では未計測。
- 代表値は**各組み合わせの最良品質試行**。実運用では試行選別や再実行のコストが乗る場合がある（Gemini 別枠は \$2.5 参照）。

AI社員が得か損か、ではなく、**同じ仕事を違う形で渡したときに何が残るか**を見ます。

4. 実測で印象的だったこと

4-1. 方式の差が、モデルの差より大きかった

品質スコア（10点満点）を、**まず同一モデル内**で見ると、方式を変えるだけで得点が大きく動きます。

表A：同一モデル内の比較（試行ごと）

モデル	試行	まるなげ	スキル	アプリ	方式差※
Composer 2.5	1	6	8	10	4
Composer 2.5	2	6	8	9	3
Composer 2.5	3	3	8	7	5

モデル	試行	まるなげ	スキル	アプリ	方式差※
Claude Opus 4.8	1	7	10	8	3
Claude Opus 4.8	2	5	6	6	1
Claude Opus 4.8	3	4	8	5	4

※方式差 = その試行の3方式のうち、最高得点 – 最低得点（=同じモデル・同じ試行で、方式だけ変えたときのブレ）

表B：同一モデル内の比較（3試行の最高点）

モデル	まるなげ最高	スキル最高	アプリ最高	方式差※
Composer 2.5	6点（試行1・2）	8点（全試行）	10点（試行1）	4
Claude Opus 4.8	7点（試行1）	10点（試行1）	8点（試行1）	3

※方式差 = 3方式の最高得点のうち、最大 – 最小

表C：モデル間の比較

試行1で横並び（同じ条件・同じ回数の試行）

方式	Composer（試行1）	Claude（試行1）	モデル差
まるなげ	6	7	1
スキル	8	10	2
アプリ	10	8	2

各方式の最良試行同士（本番判断で使った代表値）

方式	Composer 最良（試行）	Claude 最良（試行）	モデル差
まるなげ	6点（1）	7点（1）	1
スキル	8点（1～3）	10点（1）	2
アプリ	10点（1）	8点（1）	2

読み方

- 表A・B ... 同じモデルの中では、まるなげ ↔ アプリで **3～5点** 動く試行がある
- 表C ... モデルを替えても、同じ方式・同じ試行なら差はおおむね **1～2点**
- つまり今回のデータでは、「どのモデルか」より「どう渡すか」の方が、得点の振れ幅が大きい

モデル選びより、方式の選び方の方が効く——表にすると、この形です。

4-2. まるなげは「速い」こともあるが、品質点あたりでは割高

Composer まるなげは1分18秒で終わります。しかし品質6点で、字幕にない「サンクトペテルブルク勝利パレード（2025年）」のような記述が混ざりました。

速さだけ見ると魅力的ですが、**事実確認が必要な用途では、結局人間の時間が乗る**可能性が高いです。

4-3. アプリ方式は「AI社員」ではなく「AI＋仕組み」

アプリ本体（字幕取得・クリーニング・バンドル生成）は **10～12秒** でほぼモデル差なし。差はエージェントが要約する時間に出ます。

つまりボトルネックは「AI社員の頭脳」ではなく、**前処理をどこまで機械化したかと、要約に渡す情報の設計**でした。

4-4. 高単価モデルが活きる場面もある

Claude + スキルは忠実性（固有名詞・数値の正確さ）で最高でした。法務・監査・引用元として使うなら、こちらが安心です。

「高いモデル不要」は言い過ぎで、正確には **「用途によって、高いモデルが要る場面と、安いモデル＋設計で足りる場面がある」** です。

5. 経済式への当てはめ

キッカケで書いた式を、実測でざっくり当てはめるとこうなります。

実質の得 = （削減した時間 × 人間単価）
- （AIコスト）
- （検証・修正時間 × 人間単価）
- （スキル／アプリの初期設計コスト ※継続利用なら償却）

方式	AIコスト（本件）	検証コストの傾向	初期設計
まるなげ	中～高（見かけ安くても品質点あたり割高）	高い	低い
スキル	低～中	中	中（SKILL.md 作成）
アプリ	低い	低い	高（パイプライン開発）

10本目以降の継続運用なら、アプリの初期設計コストは償却されます。1本だけなら、スキルの方が手早いかもしれません。

利用枠2倍化後のざっくり試算（同一動画・最良品質帯）

構成	1本あたり（実測）	月10本（実測ベース）	2倍化後のイメージ
Composer + アプリ	47秒・約\$0.09	約8分・約\$0.9	同じ月額で 約20本分 の First-party 枠
Claude + スキル	3分41秒・約\$1.08	約37分・約\$10.8	API 枠は据え置き

※Included プラン内ではドル請求は発生しないが、「枠の消費速度」の比較として読める。
※月10本の時間合計（8分 vs 37分）も71分動画前提。短いタスクなら両方とも短くなり、倍率差も縮む（2-3 参照）。

6. 読者向けまとめ

§2（結論）と §7（提案）の要約です。冒頭の簡易結論・簡易提案と重なる部分は省略しています。

- 1. 「AI社員」=丸投げ は品質6〜7点帯で、検証コストが乗りやすい——期待外れになりやすい
- 2. **Composer 2.5 + アプリ化** なら満点も現実的。品質の争点は**方式**、時間・コストの争点は**モデル**（§2-2）
- 3. **高いモデルは不要、ではなく用途次第**——引用・監査なら Claude + スキル（§2-4）
- 4. **AIにいくら払ったか**だけでなく、**確認にいくらかかったか**も見る。コスト約1/12・時間約1/5の読み方は §2-3
- 5. **次の一步**: まるなげ → スキル化 → （繰り返すなら）アプリ化（§7）。別枠の Gemini は §2.5

道具+手順として設計すれば、期待と実測のギャップは小さくなる——今回の数字はそう示しています。

7. 提案：まるなげ → スキル化 → （繰り返すなら）アプリ化

冒頭の簡易提案のとおり、**まずスキル化、繰り返すならアプリ化**が現実的な順番です。以下は具体的な手順と、**依頼文の前に貼るプロンプト例**です。まるなげからいきなりアプリ化するのは難しい——段階を踏む前提で読んでください。

7-1. 段階のイメージ

まるなげ（丸投げ）

↓ +スキル化プロンプト（依頼文の前に足す）

スキル化（手順・品質基準を固定） ← まずここ。今日からできる

↓ 同じ仕事を何度もやると判断したら

アプリ化（機械処理をトークン外に出す） ← AIコストがさらに下がる

↓

運用手順の整理（スキル／アプリ／AIの実行方法）

段階	ハードル	今回の実測での効果
まるなげ	低い	速いが品質6〜7点。検証コストが乗りやすい
スキル化	低〜中	品質8〜10点。コード不要でまるなげから脱却
アプリ化	高い	満点・最安。繰り返し運用で効く

7-2. まずはスキル化：依頼文の前に足すプロンプト（例）

いつもの丸投げ指示の直前に、次を貼ります。そのあとに、いつも通りの依頼文を続けます。

以下の仕事の依頼文を読んで、この仕事を Cursor で繰り返し実行するためのスキル (SKILL.md) を作成してください。

スキルに含めること：

- いつこのスキルを使うか (description)
- 実行手順 (ステップごと)
- 品質チェックの観点 (人間が確認すべき点)
- 出力フォーマット
- よくある失敗と対処

依頼文に書かれていないことは推測せず、「要確認」として残してください。

使い方のイメージ:

[上のスキル化プロンプト]

[いつもの丸投げ依頼文]

例: この YouTube URL の字幕を取得して、章立て・10分セグメント要約の MD を作ってください。

これだけで、まるなげ一本より**手順と品質基準が固定された SKILL.md**ができます。以降は **@スキル名** で呼び出す——今回の実験で、スキル方式はまるなげより品質が2〜3点上がりました。

スキル化は**まるなげの依頼文をそのまま材料にできる**のがポイントです。新しい仕事をゼロから設計し直す必要はありません。

7-3. スキル化のあと：何度もやるならアプリ化へ

スキルで回し始めて、**同じ処理を何度も実行する**とわかったら、次の段階に進みます。

今回の実測では、いずれも **Composer 2.5** で測った値です。

段階	代表ファイル (試行)	品質	時間	見積コスト
スキル	skill_composer02 (試行2)	8/10	約1分00秒	約\$0.09
アプリ	app_composer01 (試行1)	10/10	47秒	約\$0.09

スキルでも約9セントと安いですが、アプリの方が **同コスト帯で満点・より速く**、前処理をトークン外に出せるぶん、長尺・多セグメントほど安定します。スキルで手順が固まっていると、「どこをコードに落とすか」も見えやすくなります。

参考: **Claude Opus 4.8** では同じ比較で、スキル **skill_claude01** (試行1) が **10点・約\$1.08**、アプリ **app_claude01** (試行1) が **8点・約\$1.32** でした。段階の話 (スキル → アプリ) はモデル共通ですが、**ドル換算の「安さ」は Composer 前提**です。

つまり **スキル化 → アプリ化** は、品質のためだけでなく **AIコストをさらに下げる** ための段階でもあります。

7-4. アプリ化：依頼文の前に足すプロンプト (例)

アプリ化に進むときは、丸投げ指示の**直前**に、次を貼ります（スキル化プロンプトの代わり、または追加）。

以下の仕事の依頼文の中で、次の3つに分けてください。

1. アプリ化できる部分
2. スキル化できる部分（手順・品質基準・プロンプトを固定すれば足りる部分）
3. AI必須の部分（要約・判断など、AIに残す部分）

アプリ化できる部分については、どのようにアプリ化すれば良いか、仕様書を作りたい。

仕様書の作り方は、ステップを分けて、小さく実装ができるようにする。

可能な限り、デバッグしやすいログを出力する。（きれいな実行結果を作らない）

ステップ単位で、実装とテストを行う。人間も想定どおりか確認する。

[] をつけて、実際に実装した部分を確認できるようにする。

仕様書はMDファイルに出力して、ステップ単位の実装・テストの都度更新していく。

スキル化できる部分については、SKILL.md の案も併記してください。

AI必須の部分については、アプリ・スキルとの入出力（渡すファイル・期待する形式）を書いてください。

そのあとに、いつもの依頼文（または既存の SKILL.md を踏まえた依頼文）を続けます。

7-5. 仕様書MDを、ステップ単位で育てる

仕様書（MD）が出力されたら、**一気に実装せず**、ステップ単位で人間が実装を指示します。

1ステップ実装 → テスト → 人間が確認 → 仕様書MDを更新（[] → [x]、ログ所見・修正メモを追記）

仕様書は一度きりの成果物ではなく、**実装とテストのたびに更新する作業台**です。アプリのステップが終わるたびに [x] を増やし、スキル案が固まったら SKILL.md を切り出し、AI必須部分の入出力も実際のファイル構成に合わせて直します。

アプリ化も一発で終わらせない。**小さく動くところまでを1ラリーにする**——まるなげだけでは品質が落ちた今回の教訓と直結します。

7-6. AI必須領域：処理は残すが、渡し方を設計し直す

アプリ化が無理な部分は、**AI処理を残します**。ただし、まるなげやスキル単体のときと同じ渡し方ではなく、**アプリ・スキルとの連携を踏まえてAI処理を考え直す**のが重要です。

- アプリが渡すもの：整形済みファイル、バンドル、デバッグログ
- スキルが固定するもの：手順、品質チェック観点、出力フォーマット

- AIがやること：渡された材料だけを読み、決められた形式で返す

「何でも考えさせる」のではなく、**前段で材料を整え、AIには判断・生成だけ任せる**——今回、アプリ+AIで満点が出た構図と同型です。

7-7. 最後にまとめるもの

スキル化・アプリ化が進んだら、運用用に次の3つをまとめます。ポイントは「**3つのドキュメント**」ではなく、**実行の順番**です。

1回の実行の流れ（本件の例）

① アプリを実行する

└ 字幕取得・整形・バンドル生成（機械処理。AIトークンは使わない）

② AIに渡すデータがそろう

└ 例：`jimaku/` のバンドル・セグメントファイル・manifest

③ スキルを使って AI に指示する

└ 例：`@youtube-jimaku-summary` + 「jimaku/ を元に要約して」

└ スキルが手順・品質基準・出力形式を固定する

④ AIが処理し、結果が出る

└ 要約テキスト（+必要ならアプリの render ステップでレポート化）

⑤ 人間が確認する

└ スキルに書いた品質観点でチェック

まるなげとの違いは、③までで **アプリが材料を整え、スキルが渡し方を固定している** ことです。AIは「URLから全部やる」のではなく、②でできたファイルだけを読んで要約する。

ドキュメントに書いておくこと

ドキュメント	いつ使うか	書く内容
アプリの実行方法	毎回、いちばん最初（①②）	コマンド、引数、出力先フォルダ、ログの見方
スキルの実行方法	アプリのあと、AIを呼ぶとき（③）	いつ @スキル名 を使うか、添える依頼文の例、品質チェック観点
AIの実行方法	③④の詳細・困ったとき	参照するファイルパス、期待する出力形式、人間が確認する点

「まるなげで終わり」ではなく、①→②→③→④→⑤が**一本の手順としてそろった状態**まで持っていく。

本リポジトリの具体例: **実行方法/実行方法_アプリ.md**（Step 1: パイプライン → Step 2: **@スキル** で要約 → Step 3: レポート生成）

7-8. 期待すること

この流れを実践すれば、AI社員が安く安定して動く状態に近づけると想定しています。

まるなげ → スキル化（まずここ） → 繰り返すならアプリ化 → AI連携の設計 → 運用手順

いきなりアプリを書かなくても、依頼文の前にプロンプトを1枚足すだけでスキル化から始められる。未検証の提案なので、次はこの手順そのものを1件試してみるのがよさそうです。

お知らせ（余談）
アプリ化をさらに進め、Cursor API で AI 処理までアプリに組み込む、という方向はあります。ただ、YouTube 字幕取得はクラウド（GitHub Actions 等）からだと拒否されやすい——本稿のパイプラインを丸ごとクラウドで自動実行する、は現実的ではありませんでした。前処理と AI を分ける、別経路で字幕を用意する、など別案はありますが、詳細は本稿では書きません。

付録

- 実験リポジトリ: 本リポジトリ（app_manunage_comp02）
- 全18試行 早見表: 付録A（品質・時間・トークン・要約メモ）
- 詳細レポート: 4_比較結果/
- 品質評価プロンプト: 要約比較プロンプト.md
- 1枚図（方式×モデル×3指標）: §2-2 の早見マトリクス表。Cursor Canvas 版は canvases/ai-comparison-matrix.canvas.tsx（未配置の場合は表のみ）

付録A. 全18試行 早見表

対象動画: koBLOf-53_g（71分27秒）
計測日: 2026-07-16
品質: 要約比較プロンプト.md の A～E 各0～2点（字幕 cleaned.txt 照合）
コスト: Cursor 公式単価 による参考見積（実測請求はすべて Included）
★ = 同一モデル×同一方式の最良品質試行（§2-2 の代表値）

品質評価の観点（A～E）

要約本文だけを、字幕 cleaned.txt を一次ソースとして評価した（要約比較プロンプト.md 準拠）。時間・トークン・コストは見えていない。

観 点	名称	何を見たか	0点の例	2点の例
A	事実忠 実性	字幕にない年号・数値・固有名 詞・因果を書いていないか（幻 覚）。不確かな箇所を「不明」や省 略にできているか	字幕にない「サンクト ペテルブルク勝利パレ ード（2025年）」を断定	817両／1379両を正し く分離。創業年欠落 を「不明」と明記
B	エンテ ィティ 一貫性	組織・製品・人物名の取り違えが ないか。英日表記を勝手に混ぜて いないか	字幕「バントール」を 「Planet等」と誤記	エアロドローム／テ レク／フレアが章を 通して安定
C	情報の 充足	結論・章立て・重要論点の抜けが ないか。10分セグメント要約の抜	セグメント欠落・後半 論点の大幅省略	4本柱+8セグメント で主要論点を網羅

観 点	名称	何を見たか	0点の例	2点の例
		け・偏り・重複はないか		
D	構造・読みやすさ	結論→章立て→セグメントの流れが明確か。冗長・重複・宣伝の扱いは適切か	セグメント間で同内容が重複、箇条書き過多	結論の4本柱圧縮が明快で実務参照向け
E	形式遵守	必須セクションの欠落がないか。「字幕によると」等のメタ文がないか。セグメント数・時間範囲が整合しているか	セグメント時間がmanifest と不整合、メタ情報混入	必須3部構成・seg00～07・時間ラベルが整合

採点: 各観点 **0～2点** (0＝問題が複数／1＝軽い問題あり／2＝良好)。同点のときは **A・B** を優先して優劣を付けた。

早見表の「観点 A～E」列 (例: **2・2・2・2・2**) は、左から A・B・C・D・E の順。

Composer 2.5 (9試行)

試行ID	方式	#	品質	観点 A～E	時間	アプリ 処理	Total Tokens	見積 \$	要約の要点・注意
まるなげ_composer01	まるなげ	1	6	0-1-2-1-2	1分 18秒	—	1,075,916	\$0.29	★同点最良。勝利パレード幻覚 (字幕に都市名・年なし)
まるなげ_composer02	まるなげ	2	6	0-1-2-2-1	1分 55秒	—	648,642	\$0.18	年号断定・砲兵4890・E300とホーネット混同
まるなげ_composer03	まるなげ	3	3	0-1-1-1-0	57秒	—	683,564	\$0.20	最速だが品質最下位。年号取り違えが複数
skill_composer01	スキル	1	8	1-1-2-2-2	55秒	—	235,102	\$0.08	Planet誤記・設立年断定など A・B で減点
skill_composer02	スキル	2	8	1-1-2-2-2	1分 00秒	—	265,614	\$0.09	★同点最良 (D・E 僅差で選出)。\$2-2 代表

試行ID	方式	#	品質	観点 A～E	時間	アプリ 処理	Total Tokens	見積 \$	要約の要点・注意
skill_composer03	スキル	3	8	1-1-2-2-2	1分 11 秒	—	295,586	\$0.10	不確実箇所の留保は良好
app_composer01	アプリ	1	10	2-2-2-2-2	47 秒	12 秒	302,962	\$0.09	★満点。817/1379 正確分離。\$2-2 代表
app_composer02	アプリ	2	9	2-2-2-2-1	40 秒	10 秒	210,647	\$0.08	設立年「字幕上不明」明示が強み
app_composer03	アプリ	3	7	1-2-2-1-1	46 秒	10 秒	206,064	\$0.08	形式・読みやすさ で減点

Claude Opus 4.8 low Thinking（9試行）

試行ID	方式	#	品質	観点 A～E	時間	アプリ 処理	Total Tokens	見積 \$	要約の要点・注意
まるなげ _claude01	まるなげ	1	7	1-1-2-2-1	4分 59 秒	—	667,467	\$1.43	★最良。物語線は 良いが 4890門・形 式ずれ
まるなげ _claude02	まるなげ	2	5	1-2-1-1-0	3分 58 秒	—	658,648	\$1.34	E形式0。カバレッ ジもやや弱い
まるなげ _claude03	まるなげ	3	4	0-1-1-2-0	4分 52 秒	—	758,892	\$1.24	年号・形式ともに 弱い
skill_claude01	スキル	1	10	2-2-2-2-2	3分 41 秒	—	501,070	\$1.08	★満点。字幕忠実 性最高。\$2-2 代表
skill_claude02	スキル	2	6	1-0-2-1-2	3分 14 秒	—	490,703	\$0.86	B一貫性で大幅減 点
skill_claude03	スキル	3	8	1-2-2-2-1	3分 38 秒	—	553,868	\$1.06	安定だが形式で減 点

試行ID	方式	#	品質	観点 A～E	時間	アプリ 処理	Total Tokens	見積 \$	要約の要点・注意
app_claude01	アプリ	1	8	1・2・2・2・1	2分 34 秒	10秒	372,597	\$1.32	★最良。戦車1379 混同が弱点。\$2-2 代表
app_claude02	アプリ	2	6	1・1・2・1・1	3分 05 秒	10秒	379,510	\$1.19	射程300km の解釈 超過
app_claude03	アプリ	3	5	0・1・2・1・1	2分 57 秒	10秒	377,603	\$1.20	第一次/第二次大戦 の混同等

方式別サマリ（全18試行）

方式	品質レ ンジ	時間レンジ (Composer / Claude)	見積\$レンジ (Composer / Claude)	傾向
まる なげ	3～7点	57秒～1分55秒 / 3分58 秒～4分59秒	\$0.18～0.29 / \$1.24～ 1.43	速いこともあるが品質・\$/点とも 最下位帯
スキ ル	6～10 点	55秒～1分11秒 / 3分14 秒～3分41秒	\$0.08～0.10 / \$0.86～ 1.08	品質安定。Composer は8点帯、 Claude は10点も出る
アプ リ	5～10 点	40秒～47秒 / 2分34 秒～3分05秒	\$0.08～0.09 / \$1.19～ 1.32	Composer で満点・最短・最安。 アプリ本体は10～12秒で固定

読み方のメモ

- 1. まるなげ_composer03 は57秒で最速だが品質3点——「速い＝得」ではない例。
- 2. まるなげ_composer01 は Total 107万トークンと最大だが、Cache Read 906k が大半（セッション再利用）。タスク固有は skill/app の 21～30万帯の方が代表に近い。
- 3. アプリ方式の壁時計時間のうち **10～12秒** はパイプライン固定。残りがエージェント要約時間。
- 4. 品質の詳細根拠・字幕照合ログは [4_比較結果/比較結果_composer/composer01～03.md](#) および [4_比較結果/比較結果_claude/claude01～03.md](#) を参照。

草案作成: 2026-07-17 / 実測データ出典: 2026-07-16 比較レポート / 追記: 2026-07 Cursor First-party 利用率2
倍化